

Número de Orden: 22

Libreta Universitaria: [REDACTED]

Nombre y Apellido: [REDACTED]

Cantidad de Hojas Entregadas: 2

Ingeniería del Software II

Parcial #2 – Análisis Estático de Programas

Ej1	Ej2	Ej3	Ej4	Nota	Aprobado/Desaprobado
B	B ⁺	B	B	9	A

R+

El examen es a libro abierto. Agregar nombre y apellido y LU a cada hoja.

Cada ejercicio se evaluará como **Bien**, **Regular** o **Mal**. Para aprobar el examen es necesario tener al menos 2 ejercicios Bien y al menos 1 ejercicio Regular: Bien = 2,5p, Regular = 1,25p, Mal = 0p.

Ejercicio 1

Se busca realizar un análisis de dataflow para predecir el estado de una variable de tipo Queue (cola), que representa colas no acotadas.

Se extiende el lenguaje TIP para incluir las expresiones: `crearQueue()` que crea una cola vacía, `enqueue(q, e)` que devuelve la cola `q` con el elemento `e` añadido al final, y `dequeue(q)` que devuelve la cola `q` sin su primer elemento (requiere que `q` no esté vacía).

Notar que `enqueue(q, e)` y `dequeue(q)` requieren que la cola `q` exista (es decir, que haya sido creada previamente con `crearQueue()`).

Por ejemplo en el siguiente programa:

```
void main() {
    var q, q1, q2, q3, q4;
    q = crearQueue(); // q esta vacía
    q1 = enqueue(q, 5); // q1 tiene un elemento
    q2 = enqueue(q1, 8); // q2 tiene dos elementos
    q3 = dequeue(q2); // q3 tiene un elemento
    q4 = dequeue(q3); // q4 esta vacía
}
```

- Se desea determinar si una cola está **no inicializada**, está **vacía**, **tiene exactamente 1 elemento** o **tiene más de 1 elemento**. Dibuja el reticulado Q que modele estos estados, indicando quien es el elemento Top ($\top$) y Bottom ($\perp$).
- Definir la semántica abstracta para las operaciones `enqueue` y `dequeue`. Tu definición debe modelar de la forma más precisa posible, pero segura (sobre-aproximada), el estado de la cola. Completa la siguiente tabla para cada elemento del reticulado que definiste en el punto anterior.

Estado Actual (q)	enqueue(q, e)	dequeue(q)
...	...	...
$\top$	...	...
$\perp$	...	...

Ejercicio 2

Para el siguiente programa Tip:

```

1  g(n) {
2      return n + 2;
3  }
4
5  main() {
6      var a, b;
7      a = 5;      // a es Impar
8      b = g(a);
9      a = 4;      // a es Par
10     b = g(a);
11     output b;
12 }
```

Se utilizará un análisis de paridad con el siguiente reticulado: **Par (P)**, **Impar (I)**, **Top (T)**, **Bottom ($\perp$)** (notar que deben definir $x = \text{cte}$, y la suma)

- Dibujar el CFG para las funciones g y main .
- Calcular el análisis de dataflow interprocedural de paridad **sin contexto de llamada**. Mostrar los valores de in y out para cada punto del programa.
- ¿Qué valor tiene $\text{out}[10](b)$? Es decir, ¿cuál es la paridad de la variable b al final de la línea 10 según este análisis?
- Recalcular el dataflow utilizando **cadenas de llamadas con $k=1$** . Indicar el valor de $\text{out}[10](b)$ según este nuevo análisis.
- Recalcular el dataflow utilizando **contexto funcional**. ¿Cuál es el valor final de $\text{out}[10](b)$?

Ejercicio 3

Dado el siguiente programa Java:

```

1  class Box { Object value; }
2
3  void main() {
4      Box b1 = new Box();      // O1
5      Box b2 = new Box();      // O2
6      Object oA = new Object(); // OA
7      Object oB = new Object(); // OB
8      b1.value = oA;
9      b2.value = oB;
10     b1 = b2;
11     Object oC = b1.value;
12 }
```

- Escribir el conjunto de restricciones de inclusión generadas por el análisis de **Andersen** para este programa.
- Encontrar la solución más pequeña (y por lo tanto, más precisa) para el sistema de restricciones que planteaste.
- Dibujar el grafo de punteros (*points-to graph*) que resulta de la solución. El grafo debe mostrar a qué objetos puede apuntar cada variable de tipo puntero y los campos de los objetos.

Ejercicio 4

Dado el siguiente programa Java:

```

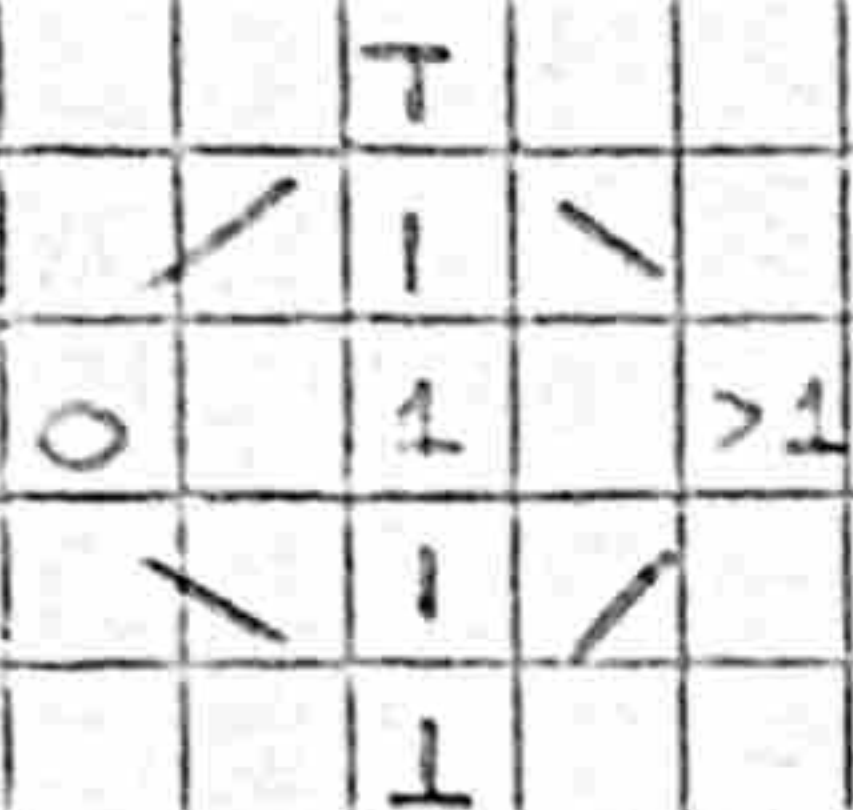
1  // Pre: { x >= 0 }
2  void main(int x) {
3      int y = 20;
4      int z = x;
5      while (z > 1) {
6          z = z - 1;
7          y = y - 3;
8      }
9  }
10 // Post: { y >= 8 }
```

- Calcular la precondition más débil (weakest precondition) $\text{wp}(\text{Programa}, y \geq 8)$. ¿Qué condición debe cumplir x para garantizar la postcondición?
- Escribir el programa que resulta de desenrollar (*unroll*) el bucle `while` una vez ($k=1$).
- Calcular la precondition más débil para el programa modificado en el punto anterior, es decir, $\text{wp}(\text{Programa_unrolled}, y \geq 8)$.

Nota: No es necesario escribir el desarrollo del cálculo de la WP en los puntos anteriores.

Ejercicio 1

a. Defino el reticulado



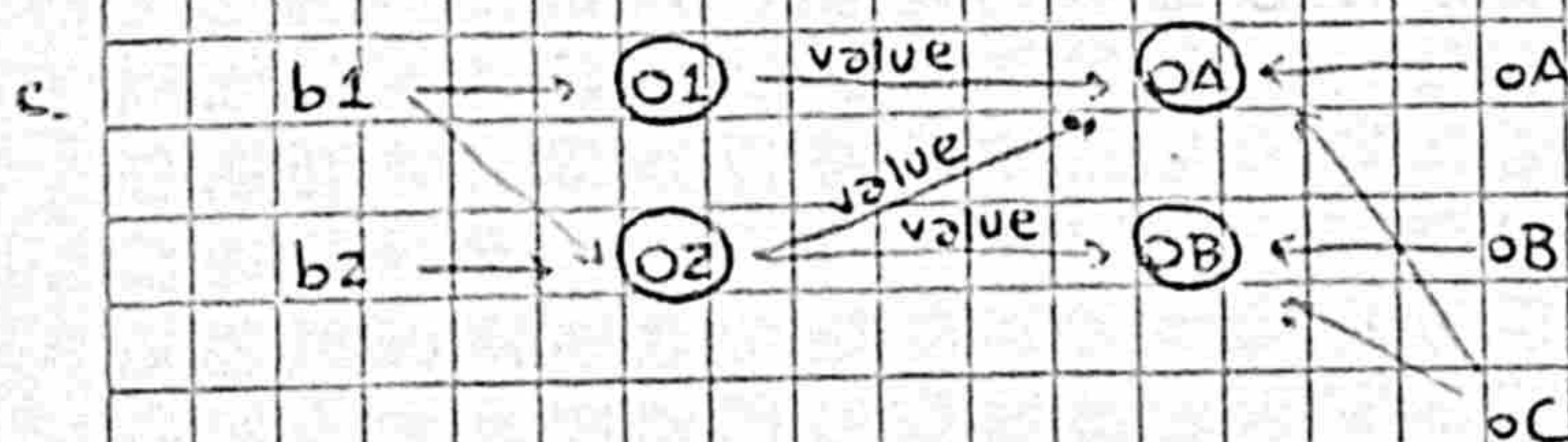
donde 0 es el estado de las colas de tamaño 0 (colas vacías), 1 es el estado de las colas de tamaño 1 y >1 es el estado de las colas de tamaño >1. Además, ⊥ es el estado de las colas no inicializadas.

q	enqueue(q, e)	dequeue(q)
⊥	⊥	⊥
1	1	1
0	>1	0
>1	>1	T

Ejercicio 3

- a.
- $\{01\} \subseteq L(b1)$
 - $\{02\} \subseteq L(b2)$
 - $\{0A\} \subseteq L(oA)$
 - $\{0B\} \subseteq L(oB)$
 - $L(oA) \subseteq \bigcap \{E(x, \text{value}) : x \in L(b1)\}$
 - $L(oB) \subseteq \bigcap \{E(x, \text{value}) : x \in L(b2)\}$
 - $L(b2) \subseteq L(b1)$
 - $\bigcup \{E(x, \text{value}) : x \in L(b1)\} \subseteq L(oC)$

- b.
- $\{01, 02\} \subseteq L(b1)$
 - $\{02\} \subseteq L(b2)$
 - $\{0A\} \subseteq L(oA)$
 - $\{0B\} \subseteq L(oB)$
 - $L(oA) \subseteq E(01, \text{value}) \cap E(02, \text{value}) \Rightarrow L(oC) = \{0A, 0B\}$
 - $L(oB) \subseteq E(01, \text{value})$
 - $L(b2) \subseteq L(b1)$
 - $E(01, \text{value}) \cup E(02, \text{value}) \subseteq L(oC)$
 - $L(b1) = \{01, 02\}$
 - $L(b2) = \{02\}$
 - $L(oA) = \{0A\}$
 - $L(oB) = \{0B\}$
 - $E(01, \text{value}, 0A)$
 - $E(02, \text{value}, 0A)$
 - $E(02, \text{value}, 0B)$



Ejercicio 4

La WP es $\{x \leq 5\}$.

Esto es así porque nos gustaría que el programa entre al ciclo a lo sumo 4 veces:

valor inicial de x	≤ 1	2	3	4	5	≥ 6
valor final de y	20	17	14	11	8	≤ 5

```

1 void main (int x) {
2     int y = 20;
3     int z = x;
4     if (z > 1) {
5         z = z - 1;
6         y = y - 3;
7         if (z > 1) {
8             assume false;
9         }
10    }
11 }

```

En el programa desenrollado se va a decrementar y a lo sumo una vez, por lo que el valor final de y va a ser 20 para $x \leq 1$ y va a ser 17 para $x > 1$. Por lo tanto, para todo valor de x se cumple la postcondición. La WP es $\{true\}$.

Ejercicio 2

Defino las funciones de transferencia para la asignación y la suma:

• Asignación: $x := n$

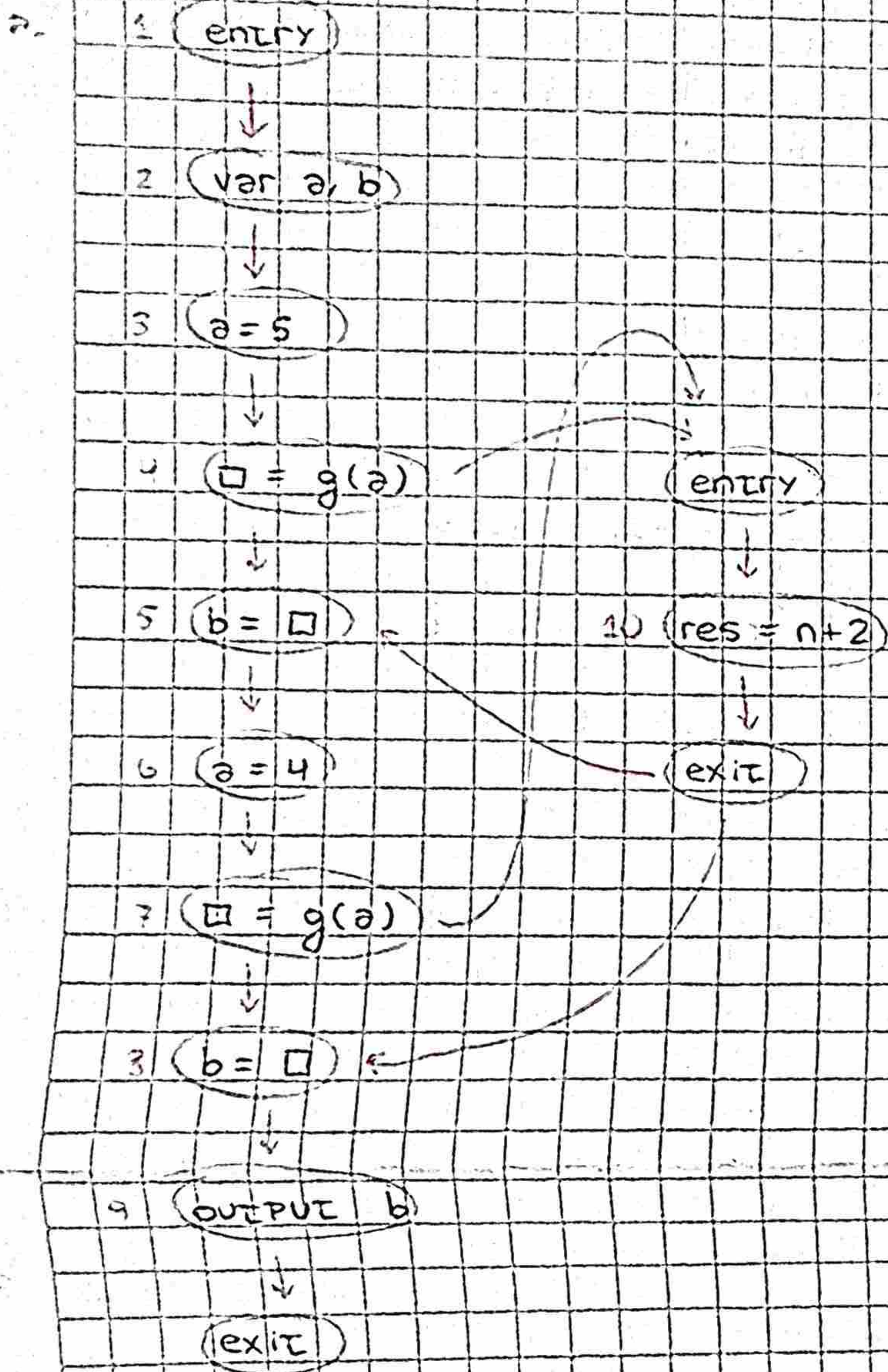
$$out[x] = \begin{cases} P & \text{si } n \% 2 == 0 \\ I & \text{si } n \% 2 != 0 \end{cases}$$

• Suma: $\tilde{x} := x + n$

$$out[\tilde{x}] = \begin{cases} \text{[scribbled out]} \end{cases}$$

super
tachado

$$\begin{cases} In[x] & \text{si } n \% 2 == 0 \\ P & \text{si } In[x] == I \wedge n \% 2 != 0 \\ I & \text{si } In[x] == P \wedge n \% 2 != 0 \end{cases}$$



b.

i	in	out
1	-	$[a \rightarrow I, b \rightarrow I]$
2	$[a \rightarrow I, b \rightarrow I]$	$[a \rightarrow T, b \rightarrow T]$
3	$[a \rightarrow T, b \rightarrow T]$	$[a \rightarrow I, b \rightarrow \boxed{I} T]$
4	$[a \rightarrow I, b \rightarrow T]$	$[a \rightarrow I, b \rightarrow T]$
10(1)	$[res \rightarrow I, n \rightarrow I] \cup [n \rightarrow I]$	$[res \rightarrow I, n \rightarrow I]$
5(1)	$[a \rightarrow I, b \rightarrow T]$	$[a \rightarrow I, b \rightarrow I]$
8(1)	$[a \rightarrow I, b \rightarrow I]$	$[a \rightarrow P, b \rightarrow I]$
6	$[a \rightarrow I, b \rightarrow I]$	$[a \rightarrow P, b \rightarrow I]$
7	$[a \rightarrow P, b \rightarrow I]$	$[res \rightarrow T, n \rightarrow T]$
10(2)	$[res \rightarrow I, n \rightarrow I] \cup [n \rightarrow P]$	$[a \rightarrow I, b \rightarrow T]$
5(2)	$[a \rightarrow I, b \rightarrow I]$	$[a \rightarrow T, b \rightarrow T]$
8(2)	$[a \rightarrow T, b \rightarrow T]$	$[a \rightarrow T, b \rightarrow T]$
9	$[a \rightarrow T, b \rightarrow T]$	

c. $out[10](b) = T$

2.

i	in	out
1	-	$[a \rightarrow \perp, b \rightarrow \perp]$
2	$[a \rightarrow \perp, b \rightarrow \perp]$	$[a \rightarrow \top, b \rightarrow \top]$
3	$[a \rightarrow \top, b \rightarrow \top]$	$[a \rightarrow \perp, b \rightarrow \top]$
4	$[a \rightarrow \perp, b \rightarrow \top]$	$[a \rightarrow \perp, b \rightarrow \top]$
10(1)	$[res \rightarrow \perp, n \rightarrow \perp] \rightarrow [res \rightarrow \perp, n \rightarrow \perp]$	$[res \rightarrow \perp, n \rightarrow \perp] \rightarrow [res \rightarrow \perp, n \rightarrow \perp]$
5	$[a \rightarrow \perp, b \rightarrow \top]$	$[a \rightarrow \perp, b \rightarrow \perp]$
6	$[a \rightarrow \perp, b \rightarrow \perp]$	$[a \rightarrow \perp, b \rightarrow \perp]$
7	$[a \rightarrow \perp, b \rightarrow \perp]$	$[a \rightarrow \perp, b \rightarrow \perp]$
10(2)	①	②
8	$[a \rightarrow \perp, b \rightarrow \perp]$	$[a \rightarrow \perp, b \rightarrow \perp]$
9	$[a \rightarrow \perp, b \rightarrow \perp]$	$[a \rightarrow \perp, b \rightarrow \perp]$

$$\textcircled{1} = [res \rightarrow \perp, n \rightarrow \perp], [res \rightarrow \perp, n \rightarrow \perp] \rightarrow [res \rightarrow \perp, n \rightarrow \perp]$$

$$\textcircled{2} = [res \rightarrow \perp, n \rightarrow \perp], [res \rightarrow \perp, n \rightarrow \perp], [res \rightarrow \perp, n \rightarrow \perp]$$

out(10)(b) = P ✓